

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL 1- Constraint-Based Problem Solving
Weightage:	40%	Total Marks:	25 50
CO5 Statement:	Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.		
Student Name:	R. DINASH KUMAR	Reg. No.:	1923122SP
Department:	B. E C E	Date:	26/08/26

ANNEXURE A
Constraint-Based Problem Solving

1. Construct an I/O communication mechanism for a real-time embedded system with the following constraints:

- CPU utilization must be below 15%
- Multiple sensors generate interrupts every 1 ms
- Use vectored interrupt handling
- Select between memory-mapped and I/O-mapped I/O for optimal latency

2. Design an I/O subsystem under these constraints:

- High-speed storage requires throughput > 3 GB/s
- Must use NVMe over PCIe
- CPU should not handle bulk data transfer
- Integrate DMA controller with interrupt notification

3. Construct a multi-device communication architecture with constraints:

- 6 peripherals using USB, 2 high-speed devices using Thunderbolt
- Avoid interrupt starvation
- Use hardware and software interrupts
- Ensure fair bus arbitration

4. Design a data acquisition system with constraints:

- Continuous data stream at 500 MB/s
- Minimal CPU intervention required
- Must compare DMA transfer vs interrupt-driven I/O
- Use memory-mapped I/O for device registers

5. Construct an interrupt handling mechanism with constraints:

- Support nested interrupts
- Maximum interrupt latency < 10 μ s
- Use vectored interrupts for high-priority devices
- Use software interrupts for background tasks

Code of Conduct:

I RDINESH kumar (192312258) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: RDINESH kumar

MARKS ALLOCATION

	Problem Understanding (25%)	Constraint Analysis (25%)	Solution Development (25%)	Application of Concepts (15%)	Justification (10%)
Q1	2	2	2	1	0.5
Q2	2	2	1.5	1	0.5
Q3	2.5	2.5	1.5	1.5	1
Q4	2.5	2	2	1.5	1
Q5	2	2.5	2	1	1

RUBRICS FOR CALCULATION:

A1

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25% 2.5	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25% 2.5	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25% 2.5	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15% 1.5	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10% 1	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1- CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL1- Constraint-Based Problem Solving
Weightage:	40%	Total Marks:	25
CO5 Statement:	<i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i>		
Student Name:	R DINESH KUMAR	Reg.No.:	192312258
Department:	B.E ECE	Date:	26/08/2026

ANNEXURE A

Constraint-Based Problem Solving

- ❖ **1. Construct an I/O communication mechanism for a real-time embedded system with the following constraints:**
- ❖ CPU utilization must be below 15%
- ❖ Multiple sensors generate interrupts every 1 ms
- ❖ Use vectored interrupt handling
- ❖ Select between memory-mapped and I/O-mapped I/O for optimal latency

A real-time embedded system has to respond to sensor inputs within a predictable amount of time. In this problem, multiple sensors generate interrupts every **1 ms**, so the processor has to handle frequent input events without allowing CPU utilization to go above **15%**. The system must also use **vectored interrupt handling** and needs a suitable choice between memory-mapped I/O and I/O-mapped I/O.

The first constraint I would consider is CPU utilization. Since interrupts can arrive every 1 ms, handling every event with a large amount of software processing could consume a significant portion of CPU time. Therefore, the interrupt service routine should be kept very short. Its main job should be to identify the source of the interrupt, read or capture the essential sensor information, clear the interrupt and then return control to the main program.

I would use **vectored interrupts** because each important sensor or interrupt source can be associated with a specific interrupt vector. When an interrupt occurs, the processor can directly jump to the corresponding service routine instead of checking every possible device to find which one generated the interrupt. This reduces interrupt-handling overhead and improves response time.

For communication between the processor and sensor registers, I would select **memory-mapped I/O**. In this approach, device registers are assigned addresses in the processor's memory address space. The CPU can access them using normal load and store instructions. For a real-time system, this gives a simple and efficient programming model and allows the same addressing mechanism to be used for memory and device registers.

The proposed mechanism would therefore be:

Sensors → Interrupt Controller → Vectored Interrupt → CPU → Sensor Register → Processing

When a sensor generates an interrupt, the interrupt controller identifies it and sends the appropriate vector to the processor. The CPU immediately enters the corresponding interrupt service routine. The routine reads the required sensor value and places it into a buffer. More complex processing can then be performed outside the interrupt routine.

Since the sensors generate interrupts every 1 ms, buffering is important. If the CPU cannot process every sensor value immediately, a small circular buffer can temporarily hold incoming data. This prevents data loss while keeping the interrupt routine short.

The **15% CPU utilization limit** should be verified by measuring the time spent servicing interrupts. If the total interrupt-processing time becomes too high, the ISR should be optimized or some processing should be moved to hardware or a background task.

Therefore, my final choice is **vectored interrupt handling with memory-mapped I/O and short interrupt service routines supported by buffering**. This design prioritizes the most important constraint—real-time response—while ensuring that CPU utilization remains below the specified 15% limit. It also follows the assessment requirement to compare memory-mapped and I/O-mapped I/O and apply interrupt-handling concepts.

❖ **2.Design an I/O subsystem under these constraints:**

- ❖ High-speed storage requires throughput > 3 GB/s
- ❖ Must use NVMe over PCIe
- ❖ CPU should not handle bulk data transfer
- ❖ Integrate DMA controller with interrupt notification

The main challenge in this problem is to build an I/O subsystem capable of achieving **more than 3 GB/s throughput** while using **NVMe over PCIe**. Another important constraint is that the CPU should not be responsible for bulk data transfer. Therefore, the design must use DMA so that large amounts of data can move directly between the storage device and main memory.

I would use an **NVMe SSD connected through a PCIe interface**. PCIe provides a high-speed connection between the storage controller and the system. NVMe is specifically designed to take advantage of high-speed PCIe communication and supports multiple queues, which is useful for parallel I/O operations.

The CPU should mainly be responsible for setting up I/O requests rather than copying every byte of data. This is where the **DMA controller** becomes important. With DMA, the CPU can provide the source or destination address, transfer size and other required parameters. The DMA hardware can then perform the actual movement of data between the NVMe subsystem and system memory.

The basic architecture would be:

CPU → NVMe Driver → PCIe/NVMe Controller → DMA → Main Memory

For a read operation, the NVMe controller obtains the requested data from the SSD and transfers it directly into a specified region of main memory. Once the transfer is complete, an interrupt can notify the CPU. The processor can then process the data without having been involved in every individual transfer.

For a write operation, the process works in the opposite direction. The required data is placed in memory and the DMA/NVMe subsystem transfers it toward the storage device.

The **greater than 3 GB/s throughput requirement** is the key performance constraint. To satisfy it, the PCIe configuration, NVMe device, DMA implementation and memory subsystem must all provide sufficient bandwidth. If one component is slower than the others, it can become the bottleneck.

Interrupt notification should also be carefully designed. Instead of generating an interrupt for every small piece of transferred data, the system can use completion notifications for larger batches of requests. This reduces interrupt overhead and allows the CPU to spend more time on useful computation.

Another advantage of NVMe is its support for multiple queues. Requests can be processed in parallel, which is useful for high-performance storage workloads. The software should therefore submit multiple requests when appropriate instead of treating the SSD like a simple low-speed device.

In my proposed design, **PCIe provides the high-speed communication path, NVMe manages storage requests, DMA performs bulk data movement, and interrupts notify the CPU after completion.** This satisfies the main constraints while avoiding unnecessary CPU involvement.

Therefore, the CPU becomes a controller of the I/O operation rather than the actual data mover. This is the most practical approach for achieving more than 3 GB/s while maintaining good overall system performance. The problem specifically requires NVMe over PCIe, DMA-based bulk transfer and interrupt notification, so these components form the core of the proposed design.

❖ **3. Construct a multi-device communication architecture with constraints:**

- ❖ 6 peripherals using USB, 2 high-speed devices using Thunderbolt
- ❖ Avoid interrupt starvation
- ❖ Use hardware and software interrupts
- ❖ Ensure fair bus arbitration

The system in this problem has **six USB peripherals and two high-speed Thunderbolt devices.** The main challenge is not simply connecting all the devices; it is ensuring that one device does not continuously occupy the communication path and prevent other devices from receiving service. Therefore, the architecture must avoid interrupt starvation and provide fair bus arbitration.

I would divide the devices according to their communication requirements. The six USB peripherals can share the USB subsystem because USB is designed to support multiple peripherals. The two high-speed devices can use Thunderbolt because they require higher-performance communication.

A simple architecture would be:

CPU / System Controller

↓

Bus / I/O Controller

USB Hub → 6 USB Devices

Thunderbolt Controller → 2 High-Speed Devices

The main constraint is **fair access to the bus**. If a high-speed Thunderbolt device continuously generates requests, lower-speed USB devices should not be completely ignored. Similarly, one USB device should not monopolize the available communication time.

To solve this, the hardware can use a fair arbitration mechanism such as round-robin scheduling. Each active device receives an opportunity to use the bus, and the arbitration logic moves to the next device after a transfer or predefined allocation. Priority can still be considered for genuinely time-critical traffic, but it should not result in indefinite blocking of lower-priority devices.

The problem also requires the use of both **hardware and software interrupts**. Hardware interrupts can be generated by devices when an important event occurs, such as data becoming available or a transfer completing. Software interrupts can be used by the operating system or software layers to request particular services or handle background communication tasks.

Interrupt starvation can occur if the processor continuously services high-frequency interrupts from one device. To avoid this, the interrupt controller and operating system should prioritize interrupts carefully. A high-priority interrupt should be handled quickly, but lower-priority devices should still receive service within a reasonable time.

For example, if one Thunderbolt device is transferring a large amount of data, the system should not allow it to block every USB device indefinitely. The bus scheduler can divide access fairly while still allowing high-speed devices to transfer data efficiently.

I would also use buffering at the controller level. If a device temporarily cannot receive service, its incoming data can be stored in a buffer until the bus becomes available. This reduces the possibility of data loss.

Therefore, my proposed architecture combines **USB for six standard peripherals, Thunderbolt for two high-speed devices, hardware interrupts for urgent device events, software interrupts for software-level services, and fair bus arbitration for balanced access**.

The most important design decision is fairness. A communication system should not only maximize speed; it should also make sure that every connected device gets an opportunity to communicate. This directly addresses the constraints given in the question and provides a practical multi-device architecture.

❖ **4.Design a data acquisition system with constraints:**

- ❖ Continuous data stream at 500 MB/s
- ❖ Minimal CPU intervention required
- ❖ Must compare DMA transfer vs interrupt-driven I/O
- ❖ Use memory-mapped I/O for device registers

The data acquisition system has to continuously receive data at **500 MB/s** while requiring minimal CPU intervention. It must also compare DMA transfer with interrupt-driven I/O and use **memory-mapped I/O** for device registers.

The most important constraint is the continuous **500 MB/s data stream**. At this rate, asking the CPU to handle every data transfer through individual interrupts would create unnecessary overhead. Therefore, I would choose **DMA as the primary data-transfer mechanism**.

In interrupt-driven I/O, the device generates an interrupt when it needs attention or when data becomes available. The CPU responds to the interrupt, reads the device register and transfers the data into memory. This approach is simple and useful for relatively low-speed devices, but at 500 MB/s it can generate a very large number of CPU operations.

DMA is more suitable because the DMA controller can transfer blocks of data directly between the acquisition device and main memory without requiring the CPU to handle each individual transfer.

The proposed structure is:

Data Acquisition Device → DMA Controller → Main Memory

****  Interrupt → CPU****

The device's control and status registers can be accessed using **memory-mapped I/O**. The CPU can configure the device by writing to these registers using normal memory access instructions. For example, the CPU can specify the buffer address, transfer size and operating mode.

Once configured, the DMA controller handles the continuous data movement. The CPU does not need to read each incoming sample. Instead, the DMA controller can transfer a complete block and generate an interrupt when the block is finished.

This dramatically reduces CPU intervention. The CPU can then process each completed block while the DMA controller fills another buffer. A **double-buffering or circular-buffer approach** would be useful here. While one buffer is being processed, another buffer can receive incoming data.

The comparison is straightforward:

Interrupt-driven I/O:

Device → Interrupt → CPU → Data transfer → Repeat

DMA:

Device → DMA → Memory → Completion Interrupt → CPU

For a 500 MB/s continuous stream, DMA is clearly more appropriate because the CPU is freed from repetitive data-copying operations. Interrupts can still be used, but they should occur at block boundaries rather than for every individual data item.

The main performance concern is maintaining enough memory bandwidth so that the incoming stream does not overwhelm the memory subsystem. The DMA controller, memory bus and destination memory must all be capable of sustaining the required rate.

Therefore, I would use **memory-mapped I/O for control registers, DMA for bulk data transfer, and interrupts only for transfer completion or exceptional conditions**. This design meets the 500 MB/s requirement while keeping CPU involvement low.

In practical terms, DMA is the better choice because the CPU should concentrate on analyzing the acquired data rather than repeatedly moving it from the device into memory. This directly satisfies all the constraints specified in the problem.

❖ 5. Construct an interrupt handling mechanism with constraints:

- ❖ Support nested interrupts
- ❖ Maximum interrupt latency < 10 μ s
- ❖ Use vectored interrupts for high-priority devices
- ❖ Use software interrupts for background tasks

The required interrupt mechanism must support **nested interrupts**, maintain a maximum interrupt latency of **less than 10 μ s**, use **vectored interrupts for high-priority devices**, and use **software interrupts for background tasks**. Since the system has a strict latency requirement, interrupt priority and handler execution time must be carefully controlled.

I would use an interrupt controller that supports multiple priority levels. High-priority devices, especially those associated with time-critical operations, would be assigned higher interrupt priority. These devices would use **vectored interrupts**, allowing the processor to directly identify the appropriate interrupt service routine.

The basic flow would be:

Device → Interrupt Controller → Priority Check → Vector → ISR → Return

When a high-priority interrupt occurs, the processor saves the necessary execution state and immediately transfers control to the corresponding interrupt service routine.

The system must also support **nested interrupts**. This means that while a lower-priority interrupt service routine is running, a higher-priority interrupt can interrupt it. The processor temporarily saves the current interrupt context, services the higher-priority event and then returns to the interrupted routine.

However, nested interrupts must be controlled carefully. If every interrupt is allowed to interrupt every other interrupt, the processor could spend too much time switching between handlers. Therefore, only higher-priority interrupts should normally be allowed to preempt a lower-priority handler.

To satisfy the **less-than-10- μ s maximum latency**, interrupt service routines should be short and efficient. The handler should perform only the time-critical operations, such as reading a status register, capturing important data, clearing the interrupt and scheduling further processing. Large computations should be moved outside the ISR.

High-priority devices should use vectored hardware interrupts because they require predictable and fast response. **Software interrupts**, on the other hand, can be used for background operating-system services or tasks that do not require the same real-time response.

Another important design feature is interrupt masking. During a critical section, certain lower-priority interrupts may temporarily be masked, but high-priority emergency interrupts should remain available whenever possible. This prevents unnecessary delays to critical events.

The system should also be tested by measuring the time between an interrupt being generated and the start of its service routine. If the measured latency approaches 10 μ s, the interrupt structure should be optimized further.

Therefore, my proposed mechanism uses **priority-based nested interrupts, vectored handling for high-priority hardware devices, short ISRs, and software interrupts for background tasks**. This provides a good balance between fast response and efficient CPU utilization.

The most important point is that meeting the 10- μ s requirement is not achieved only by choosing vectored interrupts. The complete design—including interrupt priority, context switching, ISR length and interrupt-controller behavior—must be optimized. This makes the solution more realistic for a real-time system and directly satisfies the constraints in the given assessment.

Code of Conduct:

I R DINESH KUMAR(192312258) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

DINESH KUMAR

MARKS ALLOCATION

	Problem Understanding (25%)	Constraint Analysis (25%)	Solution Development (25%)	Application of Concepts (15%)	Justification (10%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
----------	-----------	---------------------	----------------------	----------------------	---------------------

Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided